

ヒープの構築と再構築（ヒープ化）

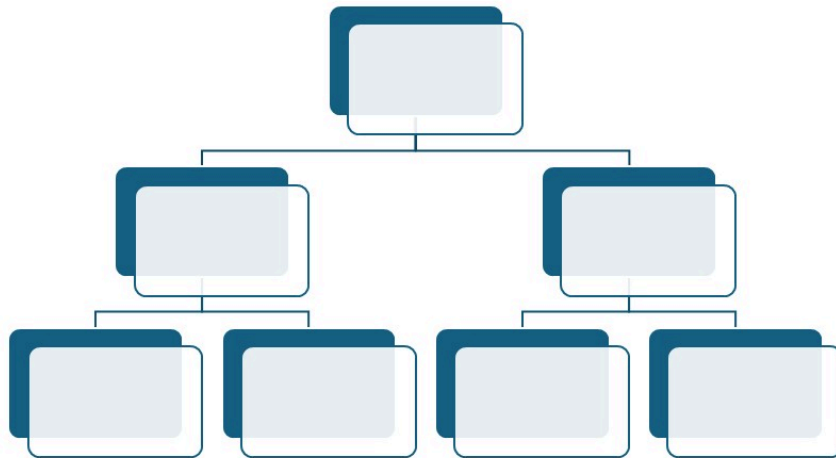
ヒープの構築

例として、[5, 2, 7, 3, 6, 1, 4] というバラバラに並んだ配列をツリーに追加して、「親が子より大きい」というルールにしたがって、ヒープを構築します。

Array

0	1	2	3	4	5	6
5	2	7	3	6	1	4

Heap



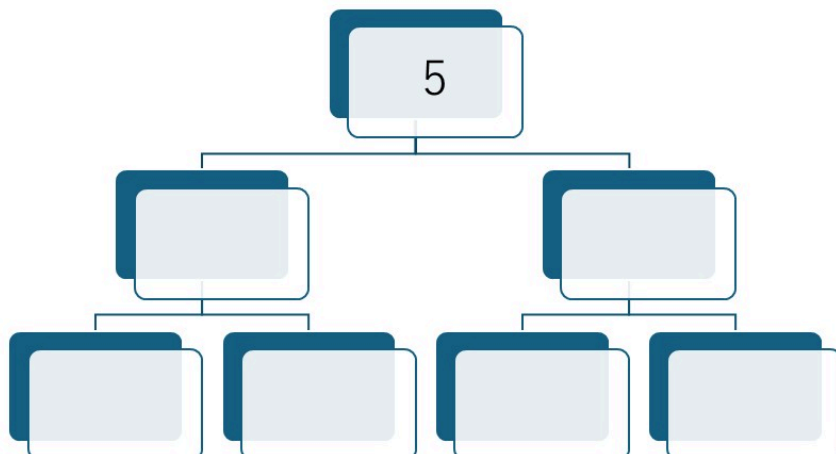
- 5 をツリーに追加

Array

0	1	2	3	4	5	6
	2	7	3	6	1	4

Heap

降順になるように
ヒープを構築する



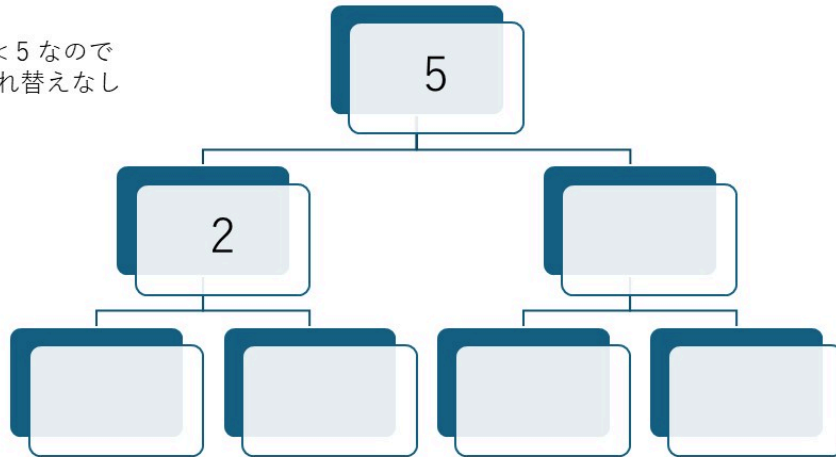
- 2 を追加 → $2 < 5$ なので、安定

Array

0	1	2	3	4	5	6
		7	3	6	1	4

Heap

$2 < 5$ なので
入れ替えなし



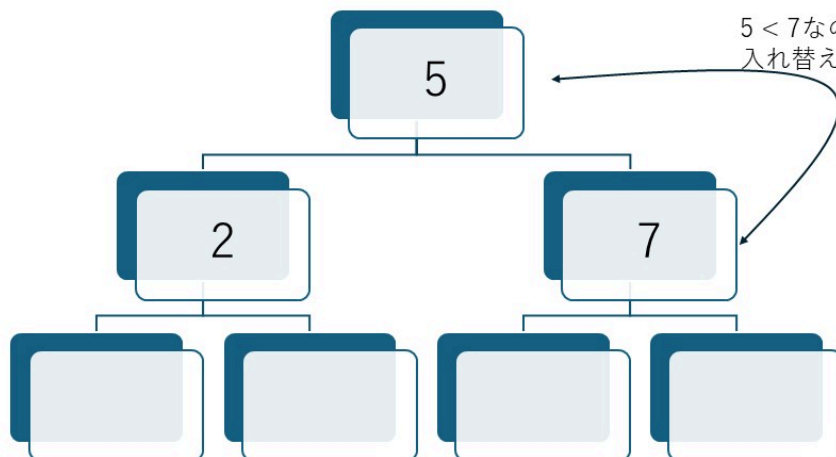
- 7 を追加 → $5 < 7$ なので、入れ替え

Array

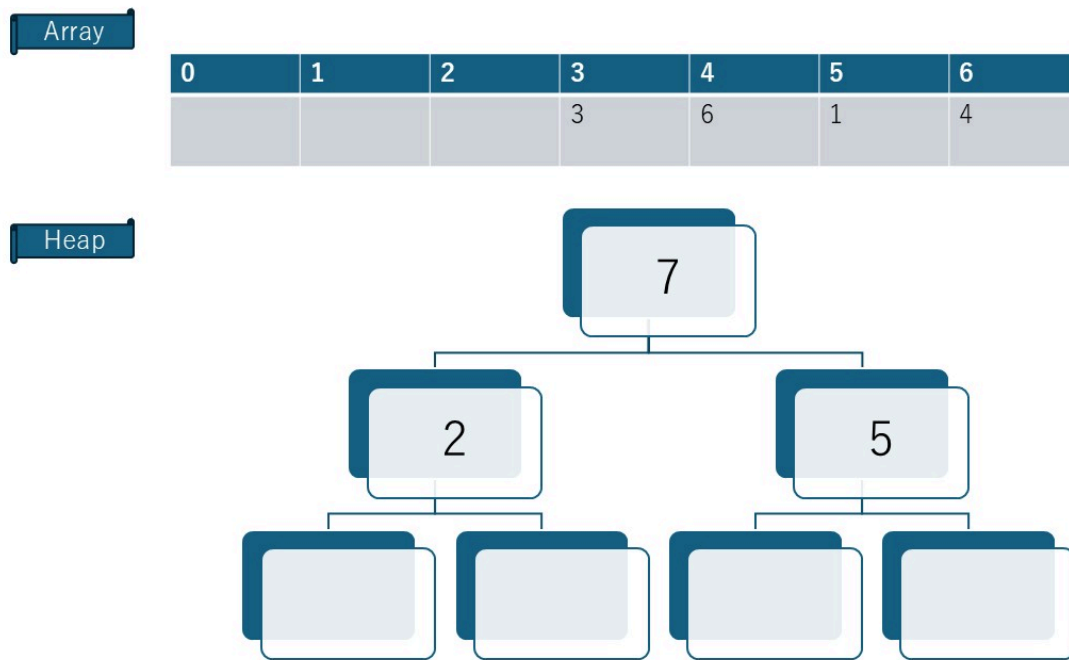
0	1	2	3	4	5	6
			3	6	1	4

Heap

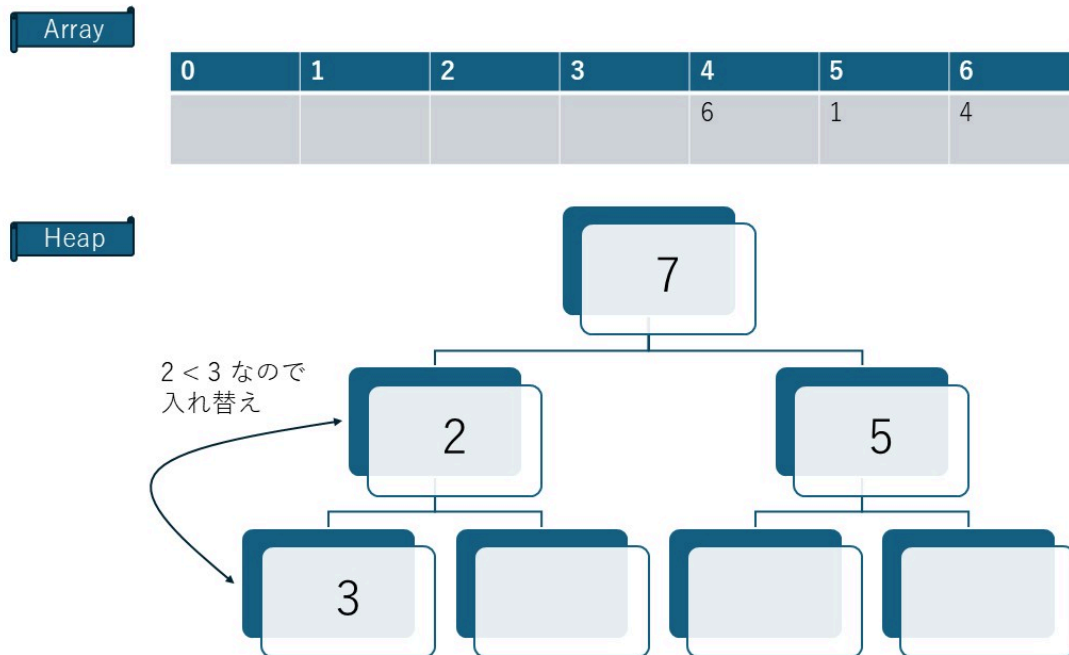
$5 < 7$ なので
入れ替え



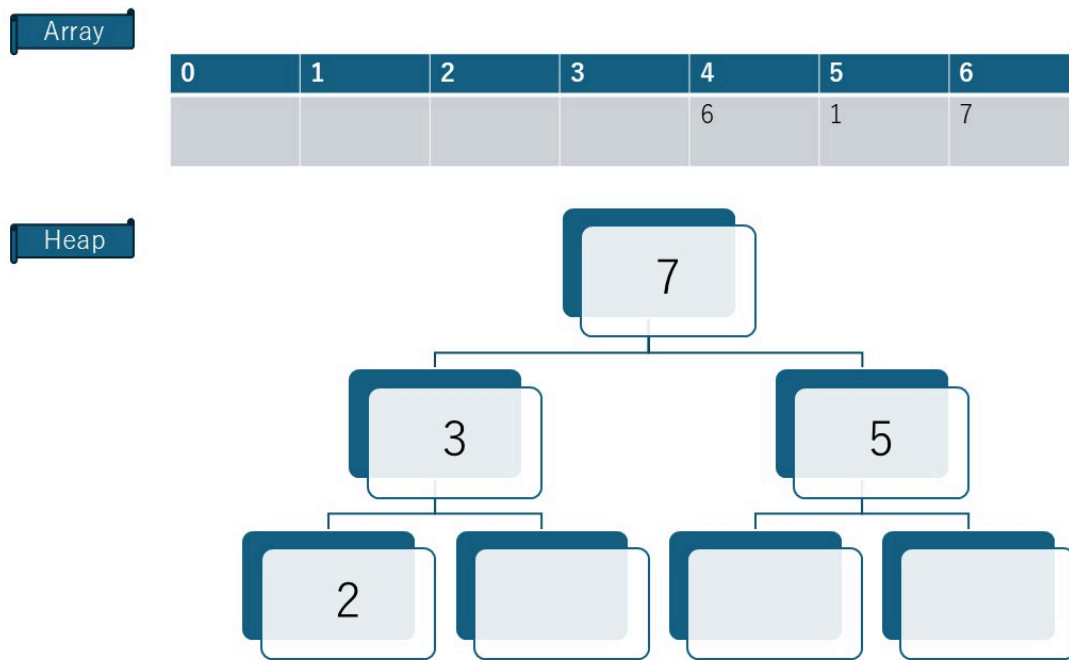
- 入れ替え後、親 > 子のルールに従っているので、**安定**



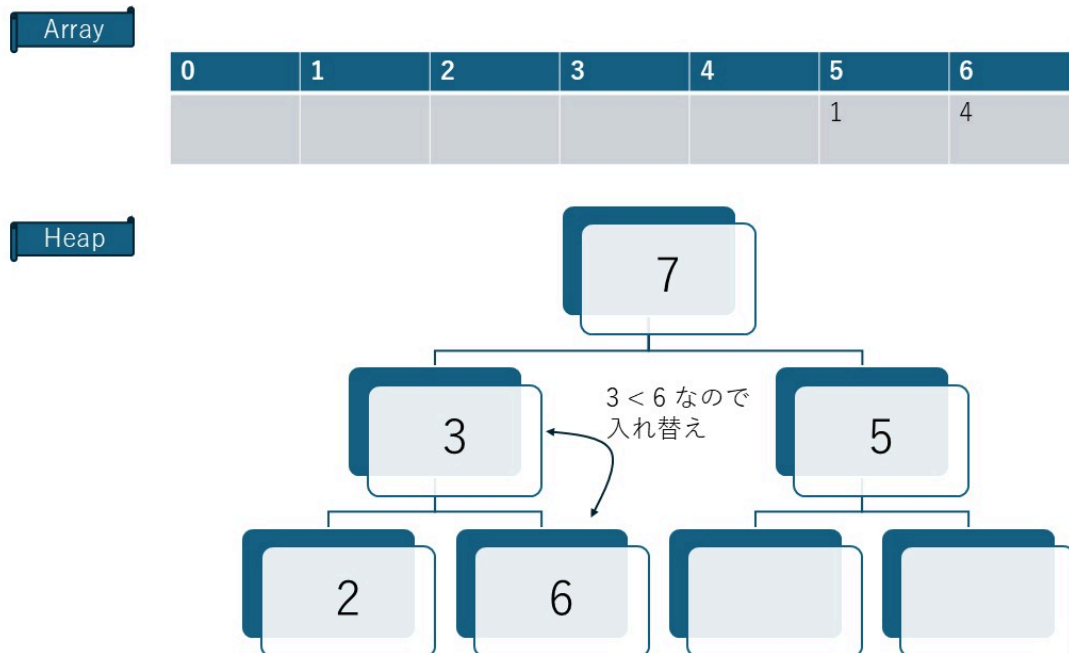
- 3 を追加 → $2 < 3$ なので、**入れ替え**



- 入れ替え後、親 > 子のルールに従っているので、**安定**



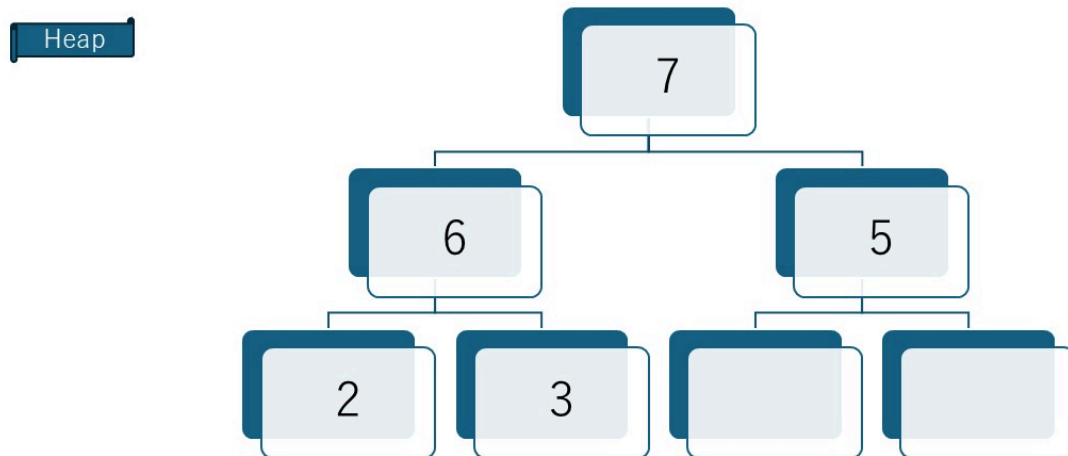
- 6 を追加 → $3 < 6$ なので、**入れ替え**



- 入れ替え後、親 > 子のルールに従っているので、**安定**

Array

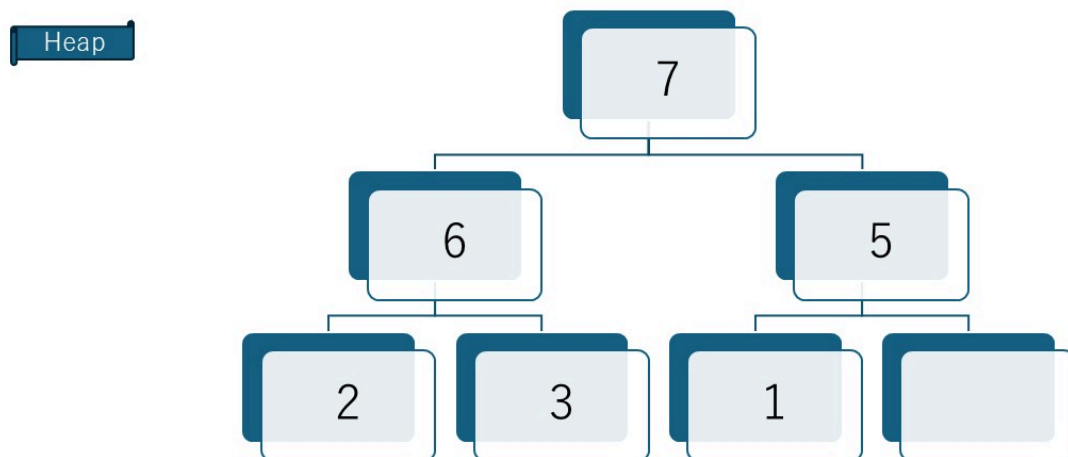
0	1	2	3	4	5	6
					1	4



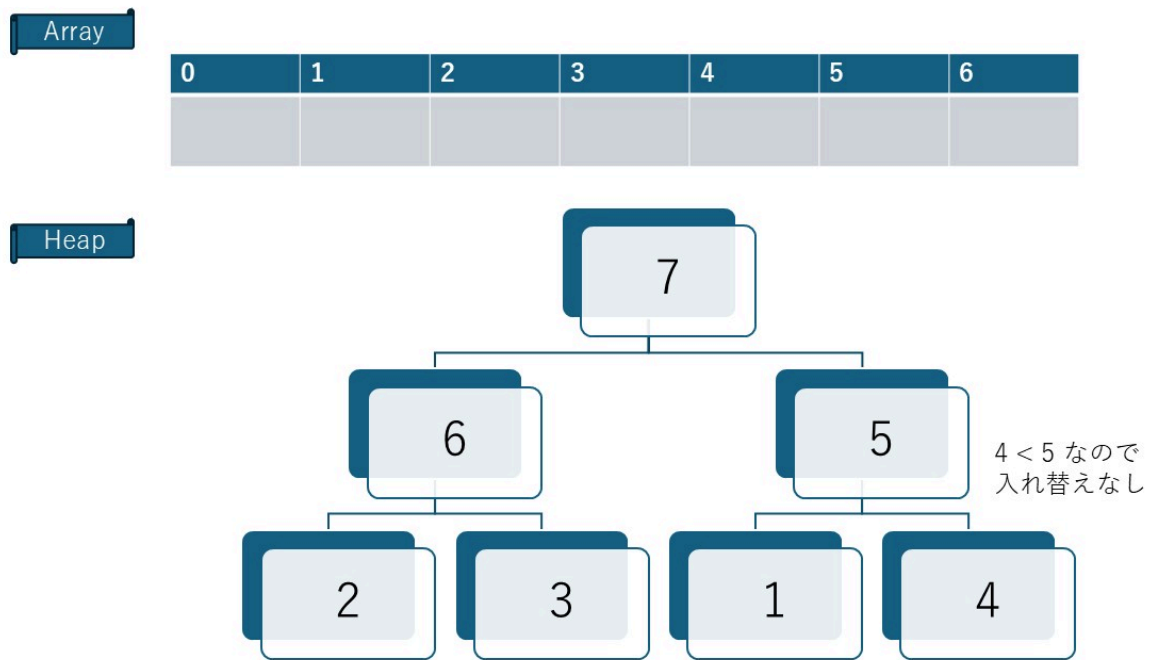
- 1 を追加 → $1 < 5$ なので、**安定**

Array

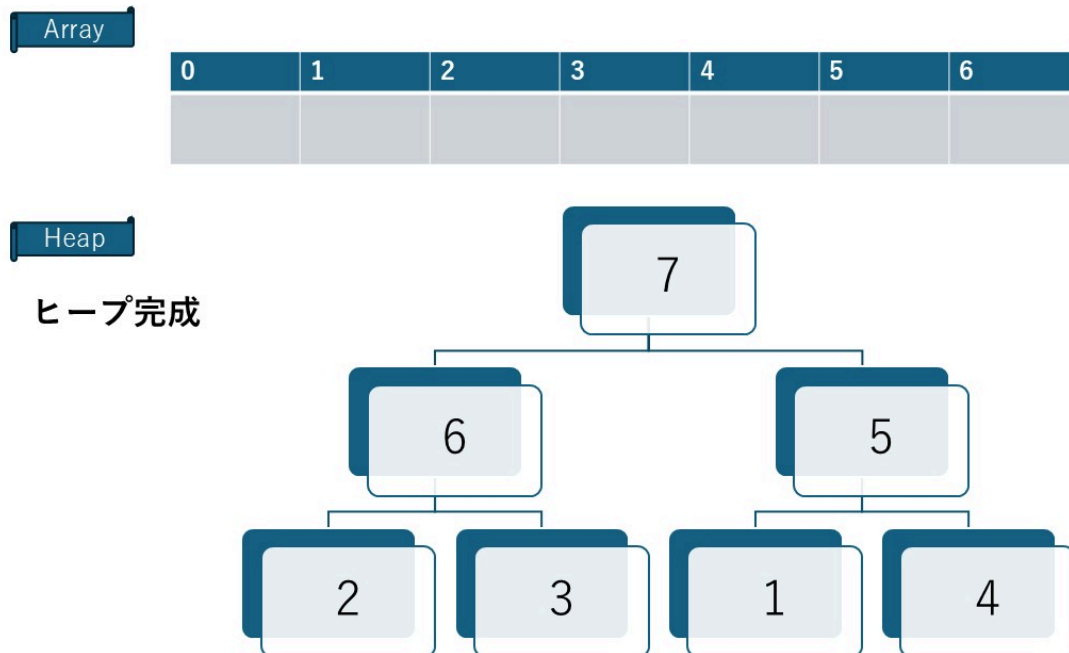
0	1	2	3	4	5	6
						4



- 4 を追加 → $4 < 5$ なので、安定



- これで、ヒープ完成!



ヒープの再構築：ヒープ化 (Heapify)

データを削除（頂点を取り出す）したとき、ヒープは以下の手順で形を整えます。

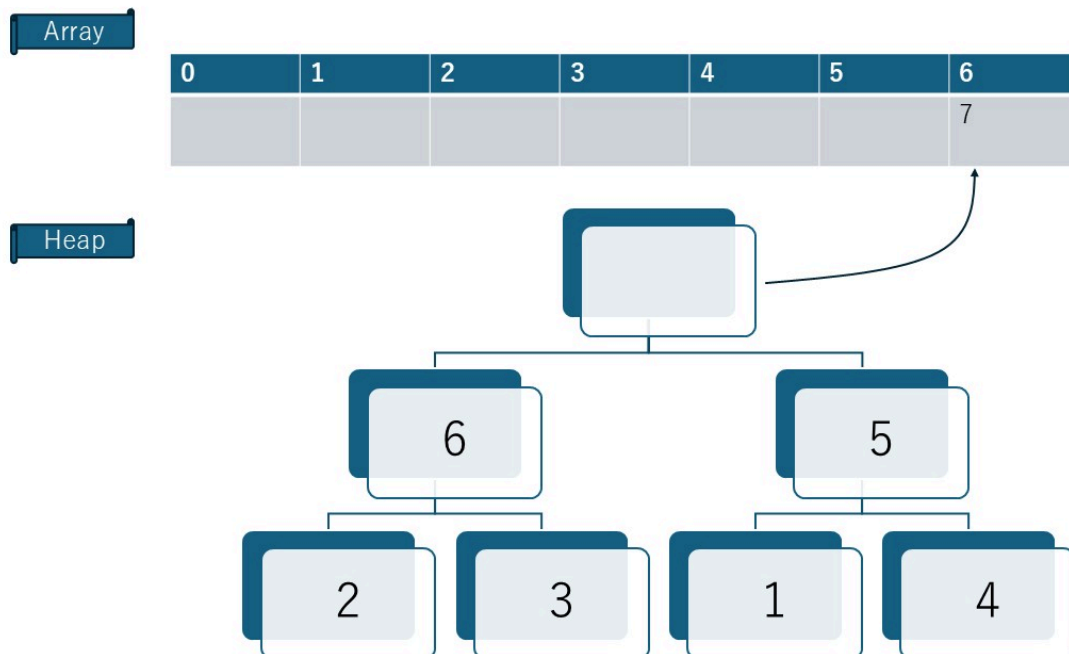
1. 空いた頂点に、一番末尾（右下）の要素を持ってくる。
2. その要素を子と比較し、ルール違反（子が自分より大きい等）なら入れ替える。
3. これを適切な位置に収まるまで繰り返す（沈み込み）。

これを使って、配列をソートすることができます（ヒープソート）。

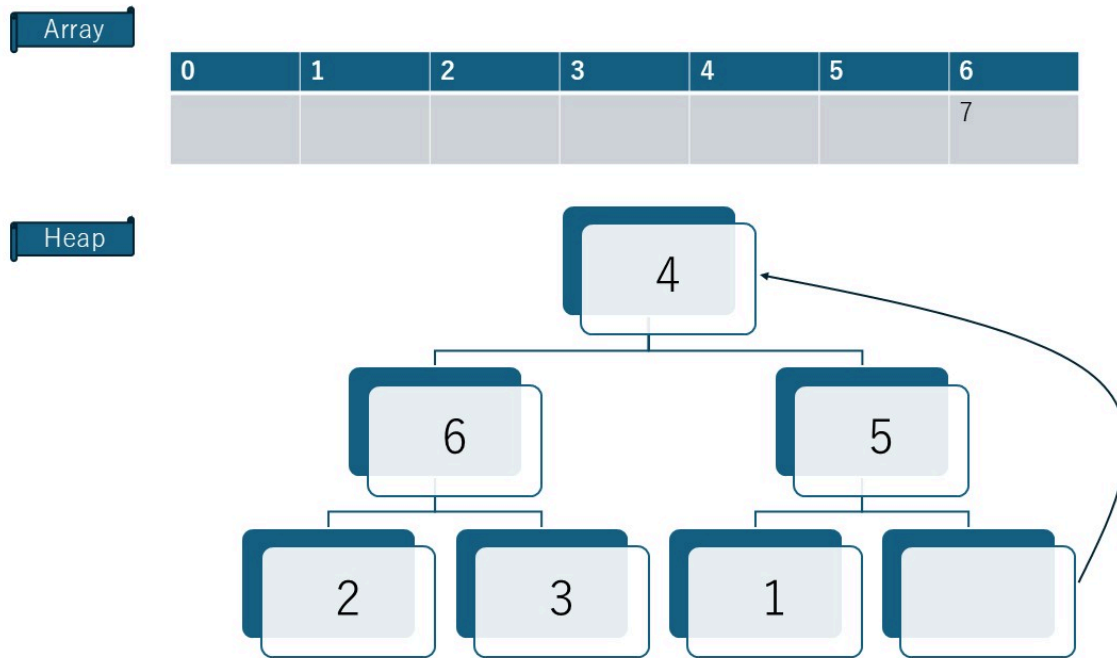
![note]

ヒープソートの実装については、「ヒープソート」の章でくわしく説明します。ここでは、「ヒープの再構築」の例として図解します。

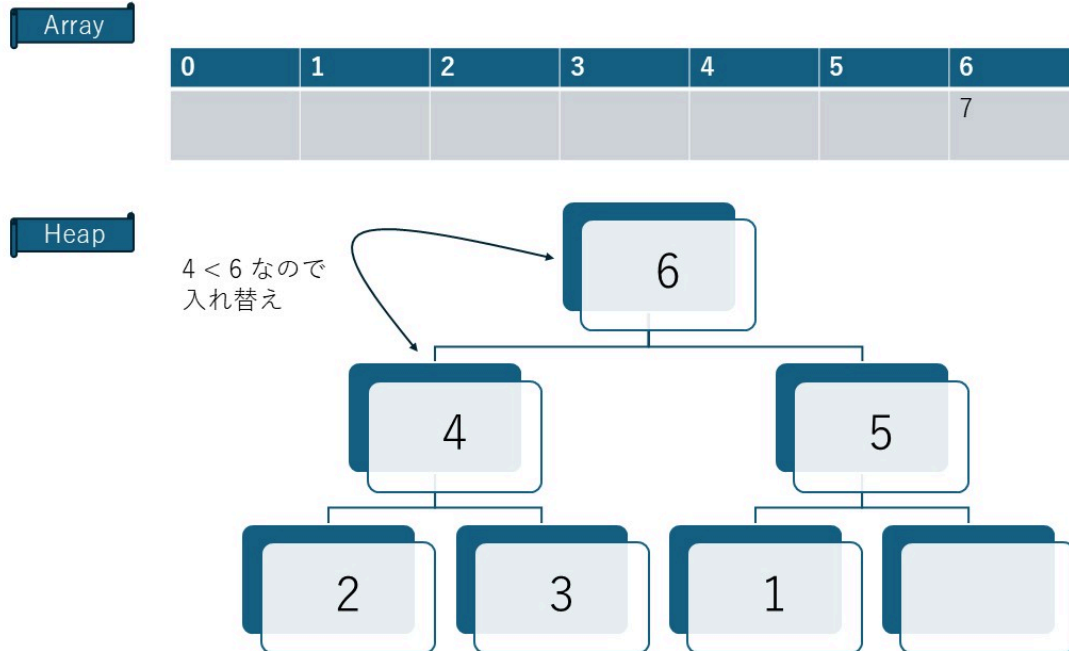
- 頂点の 7 を取り出す



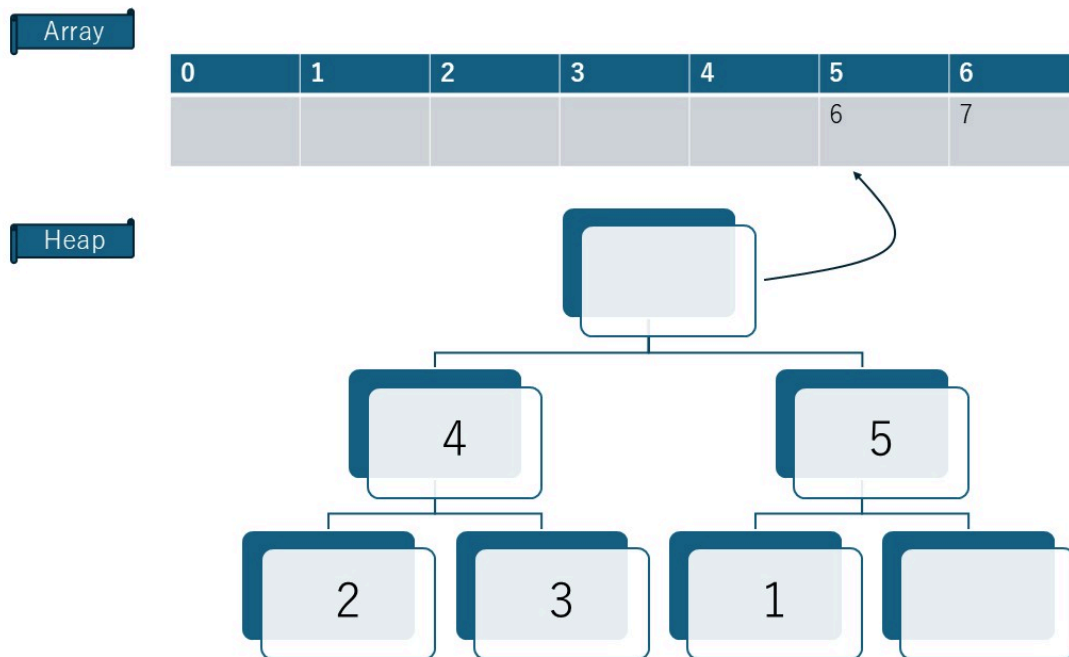
- 右下の 4 を空いた頂点に移動



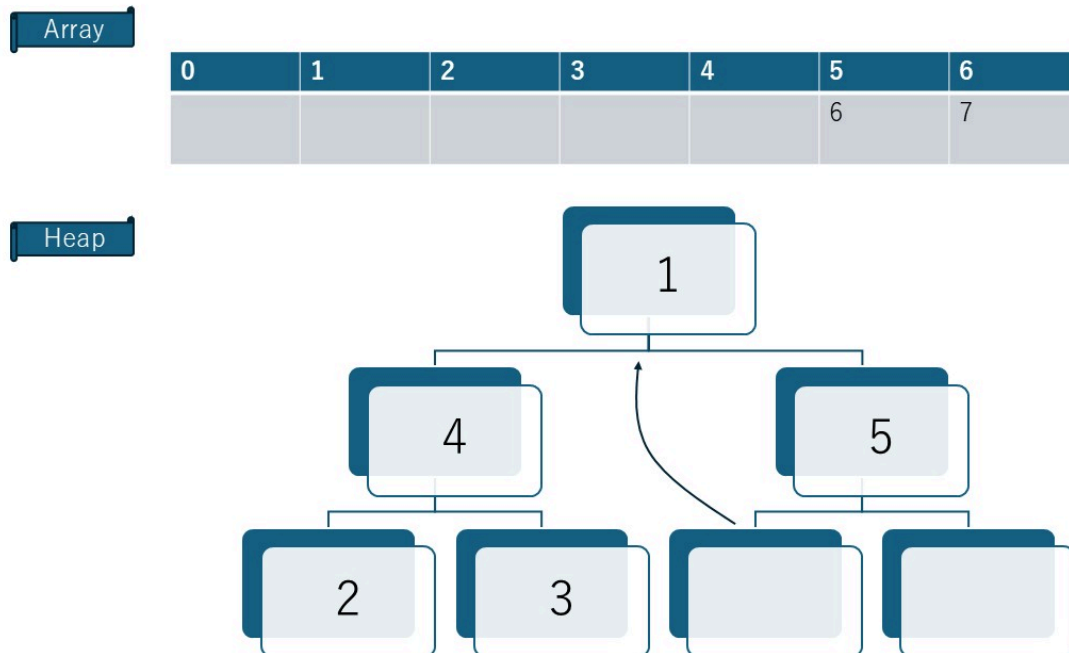
- $4 < 6$ なので、入れ替え



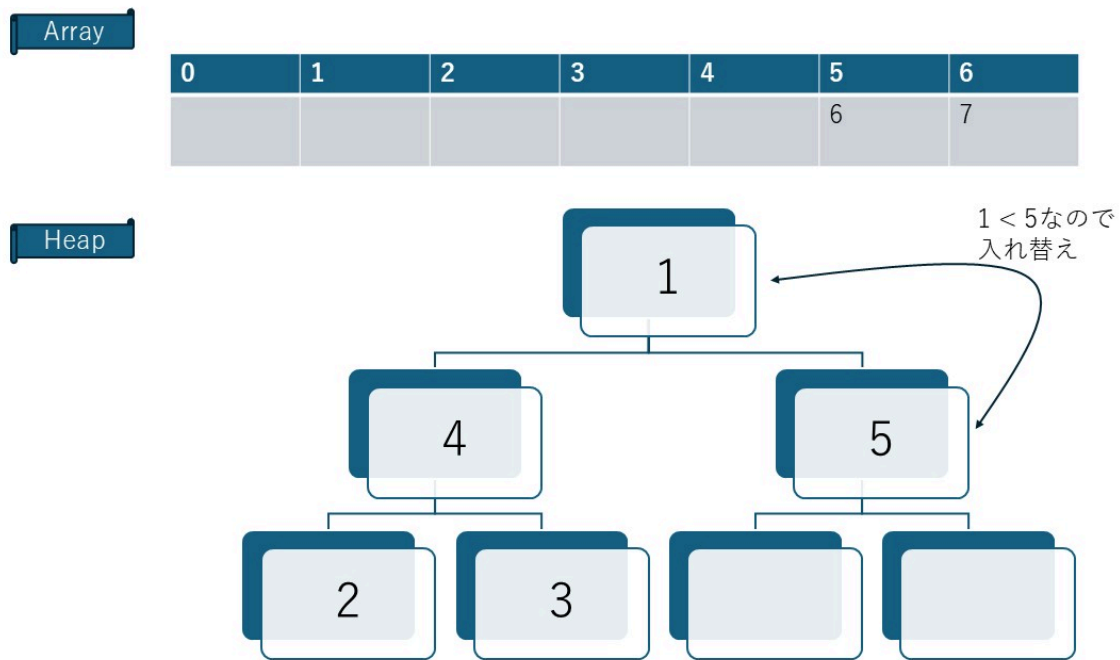
- 頂点の 6 を配列に戻す



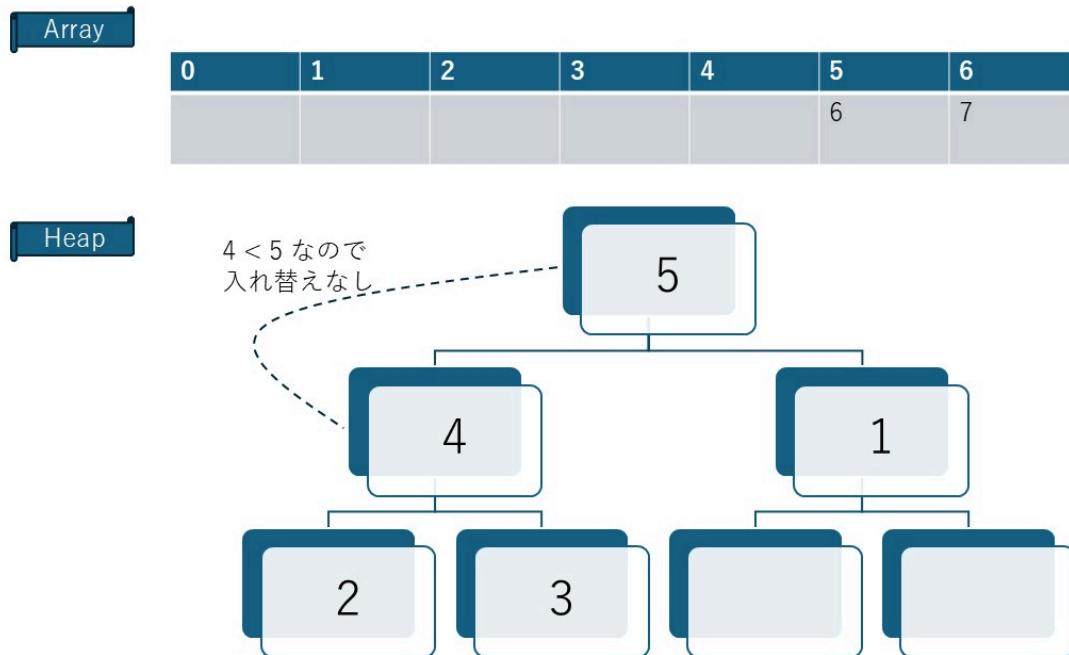
- 右下の 1 を空いた頂点に移動



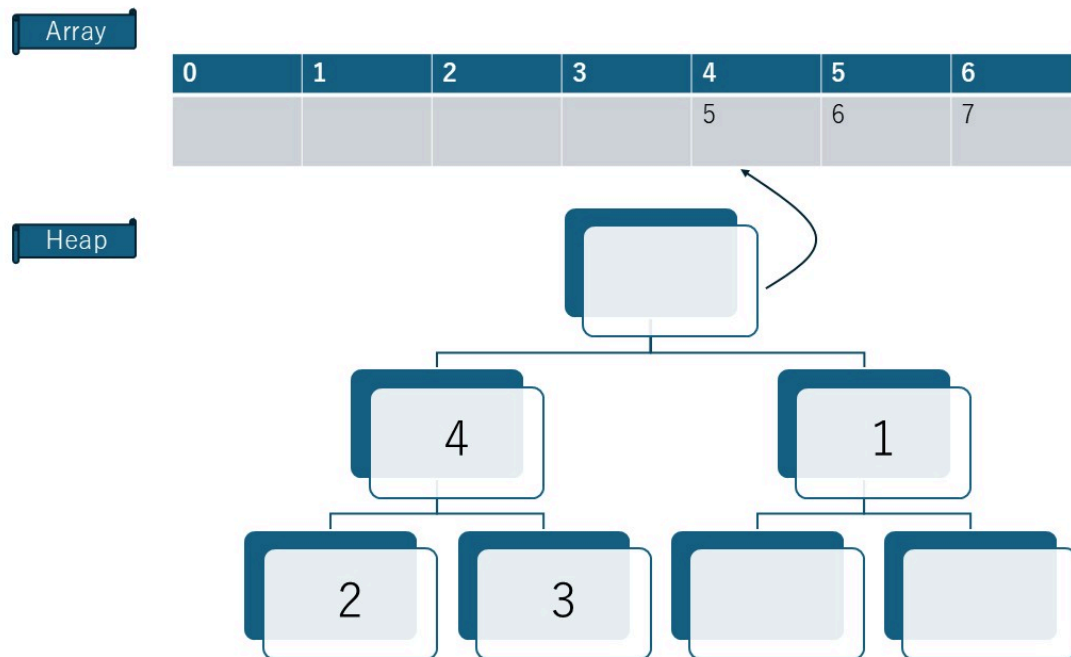
- 1 < 5なので、入れ替え



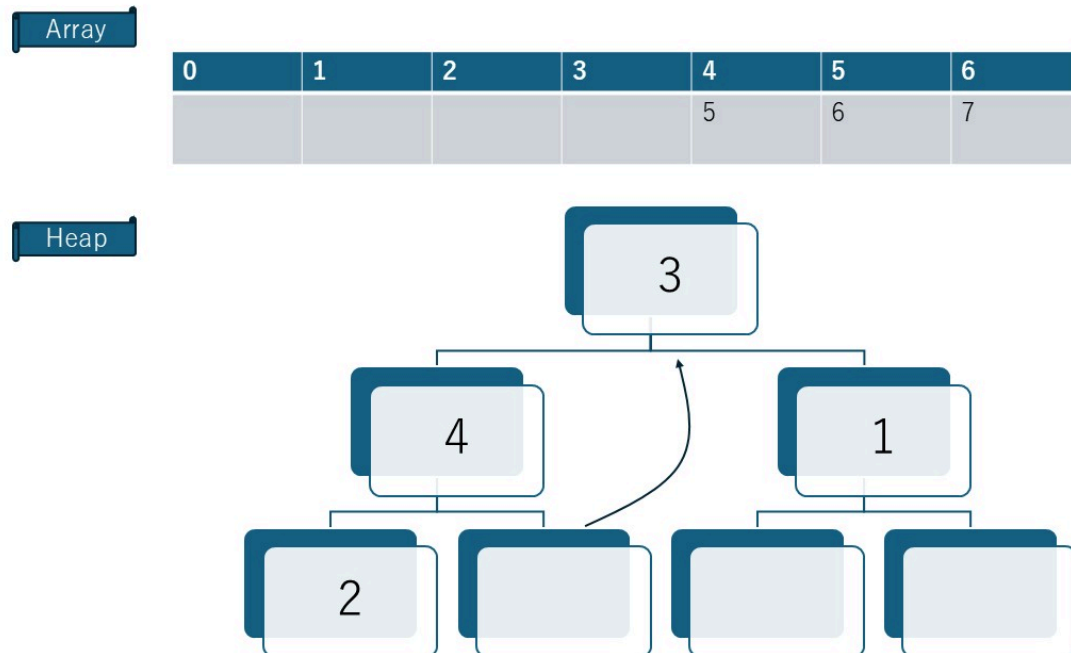
- 4 < 5なので、安定



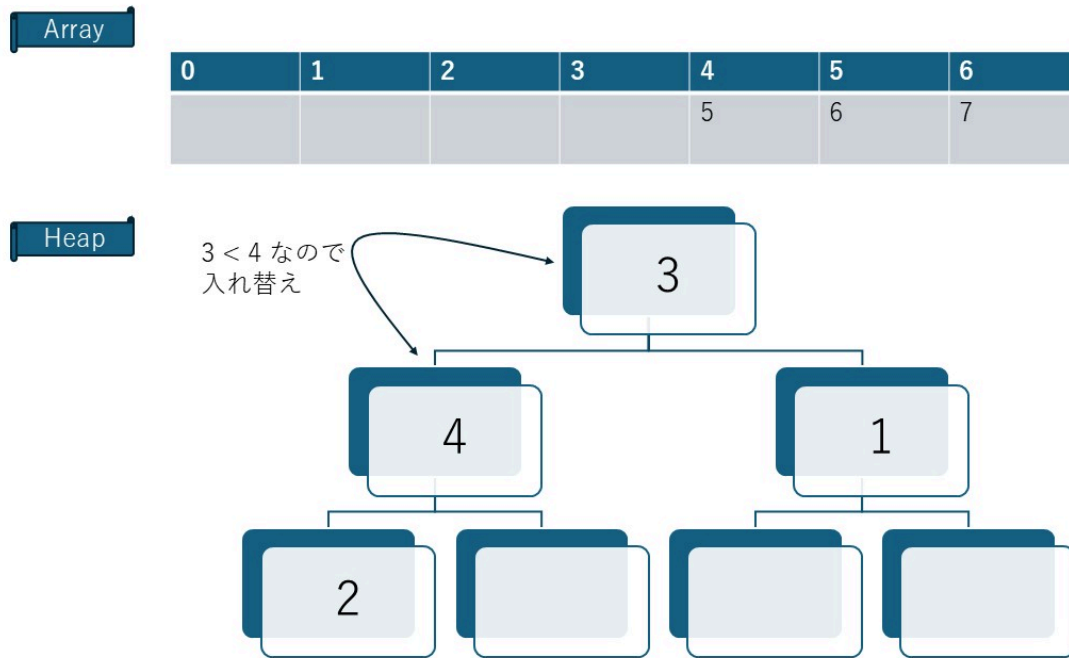
- 頂点の 5 を配列に戻す



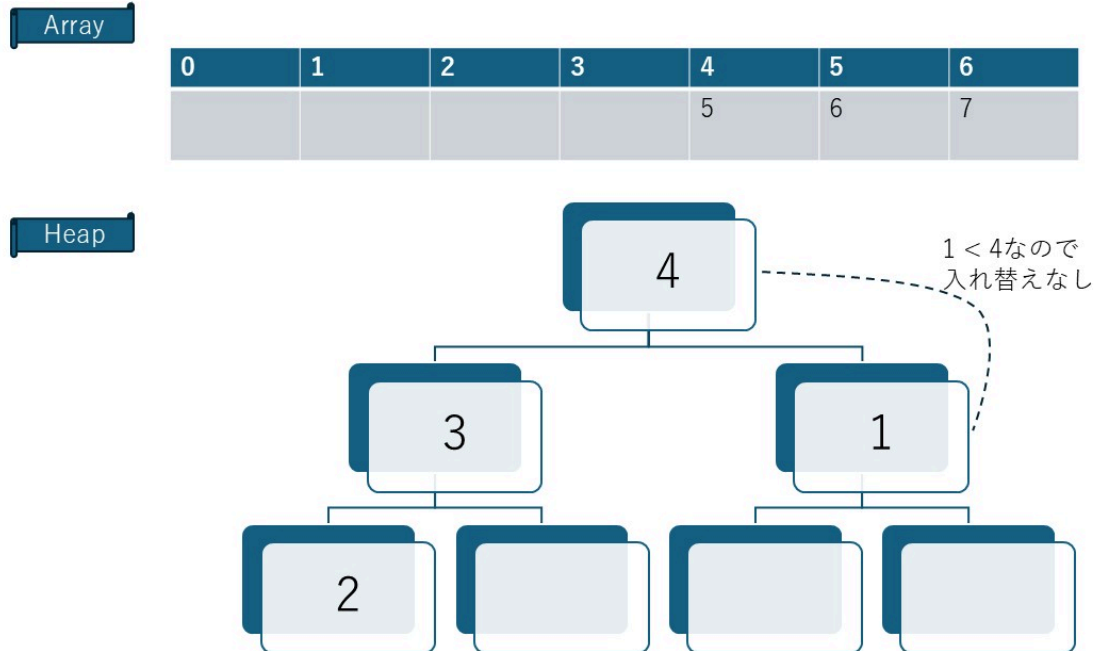
- 右下の 3 を空いた頂点に移動



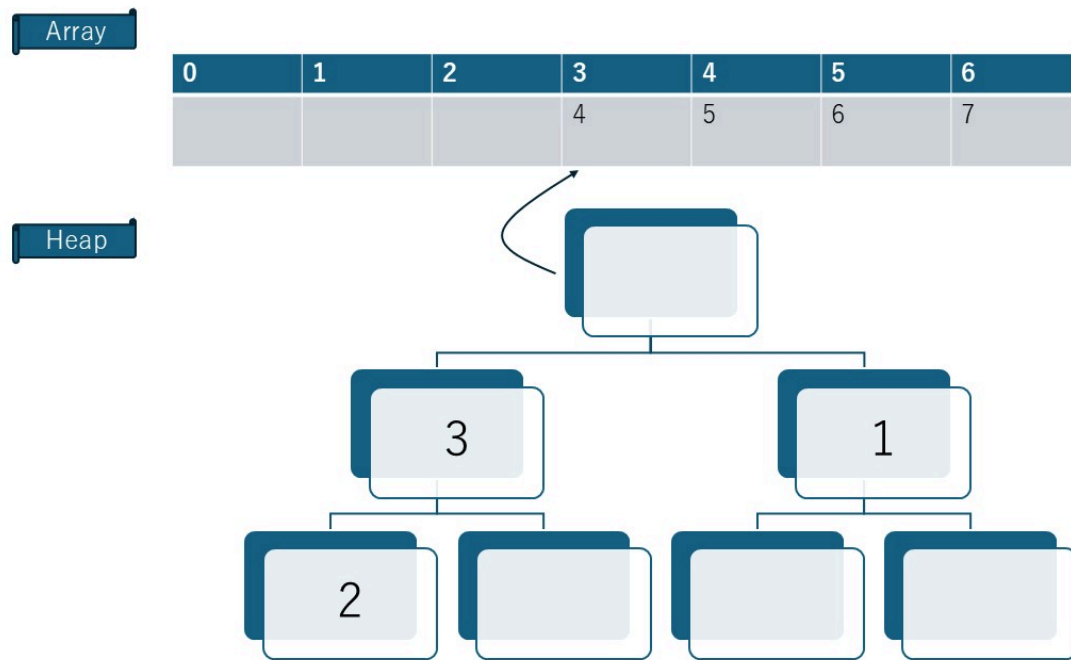
- 3 < 4 なので、入れ替え



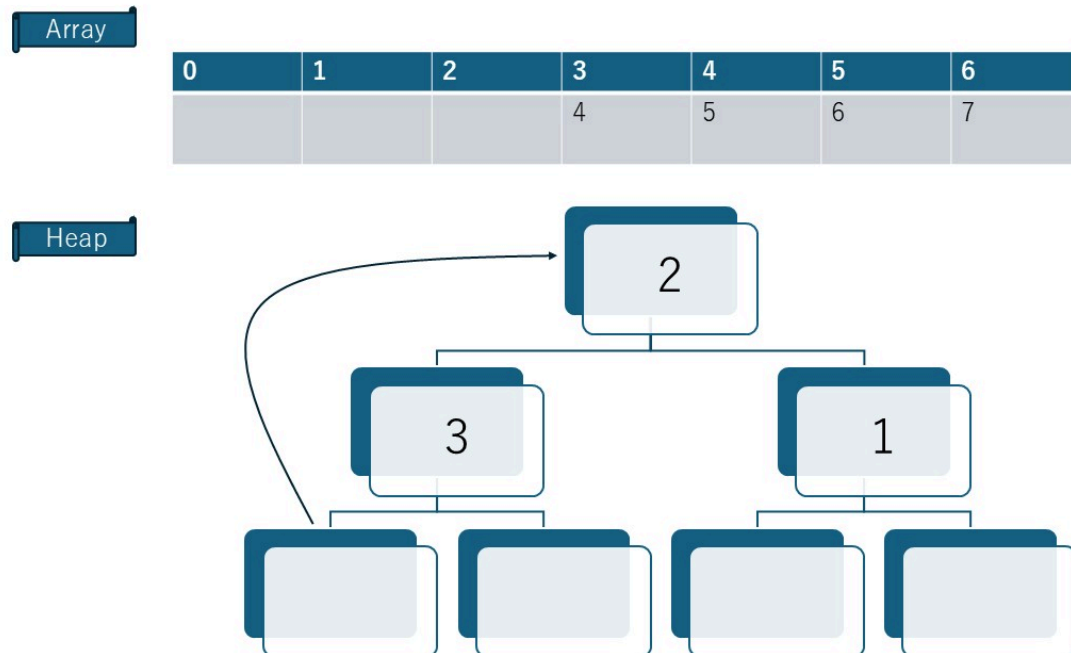
- 1 < 4 なので、安定



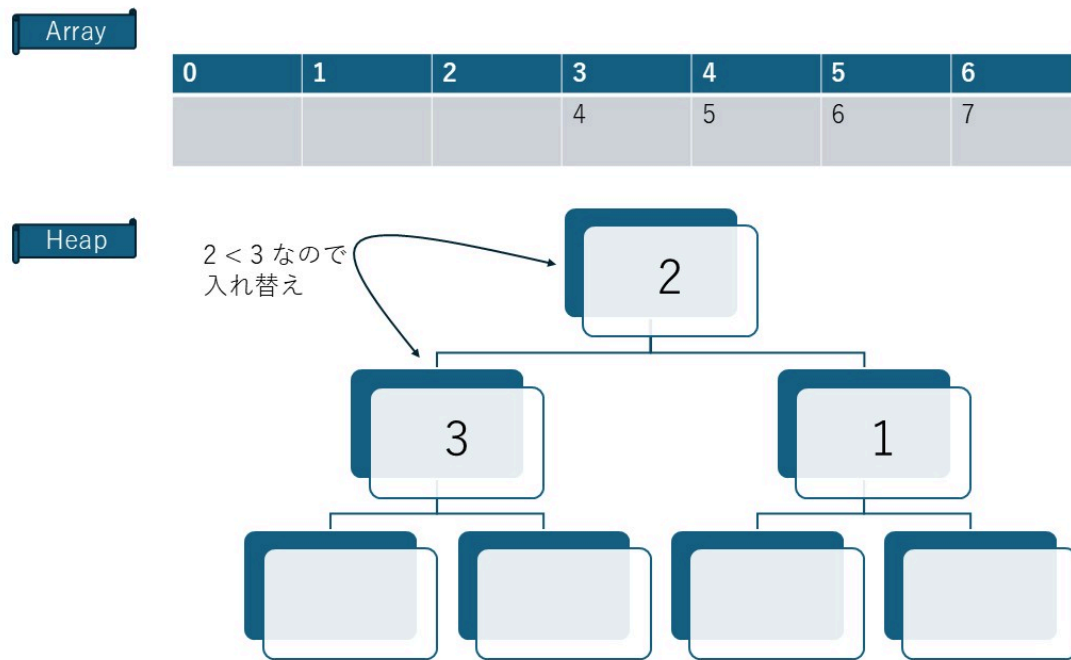
- 頂点の 4 を配列に戻す



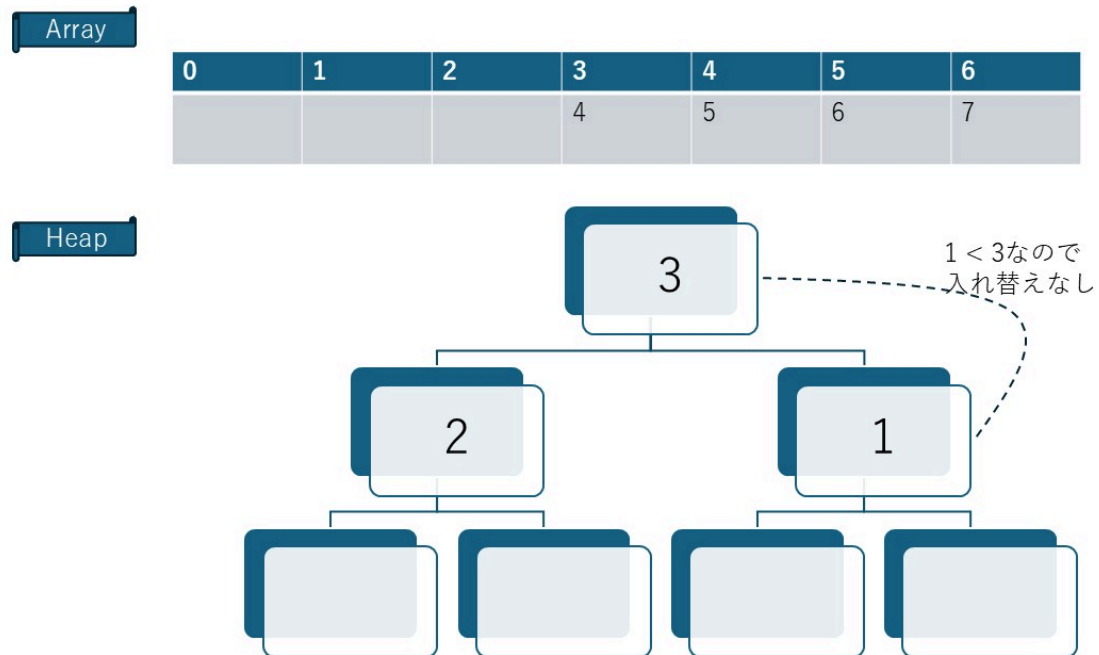
- 右下の 2 を空いた頂点に移動



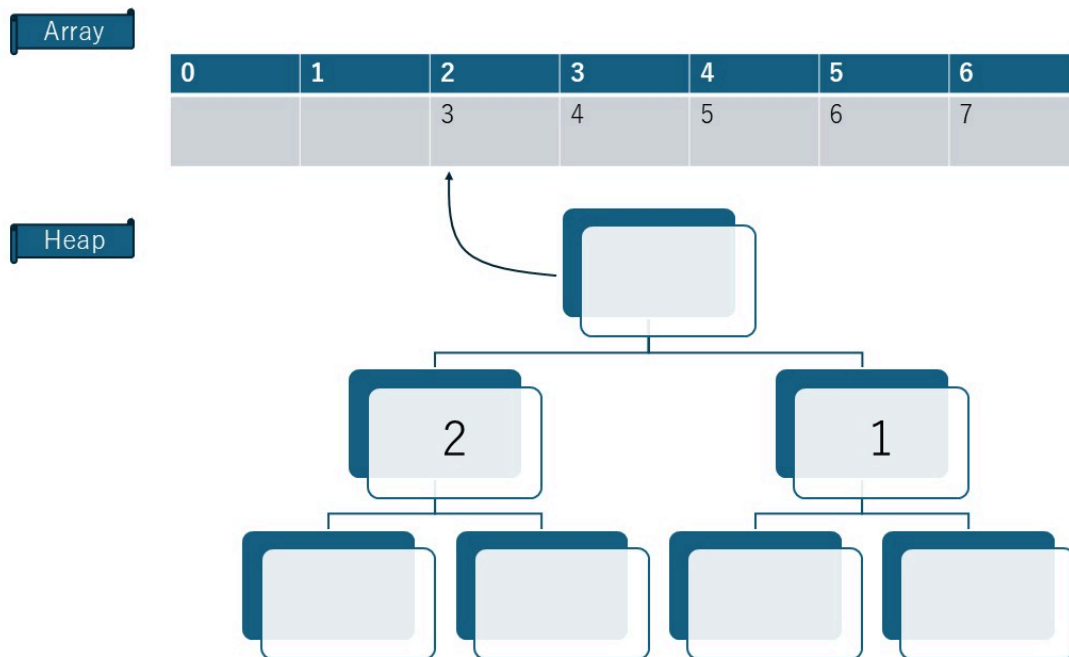
- $2 < 3$ なので、入れ替え



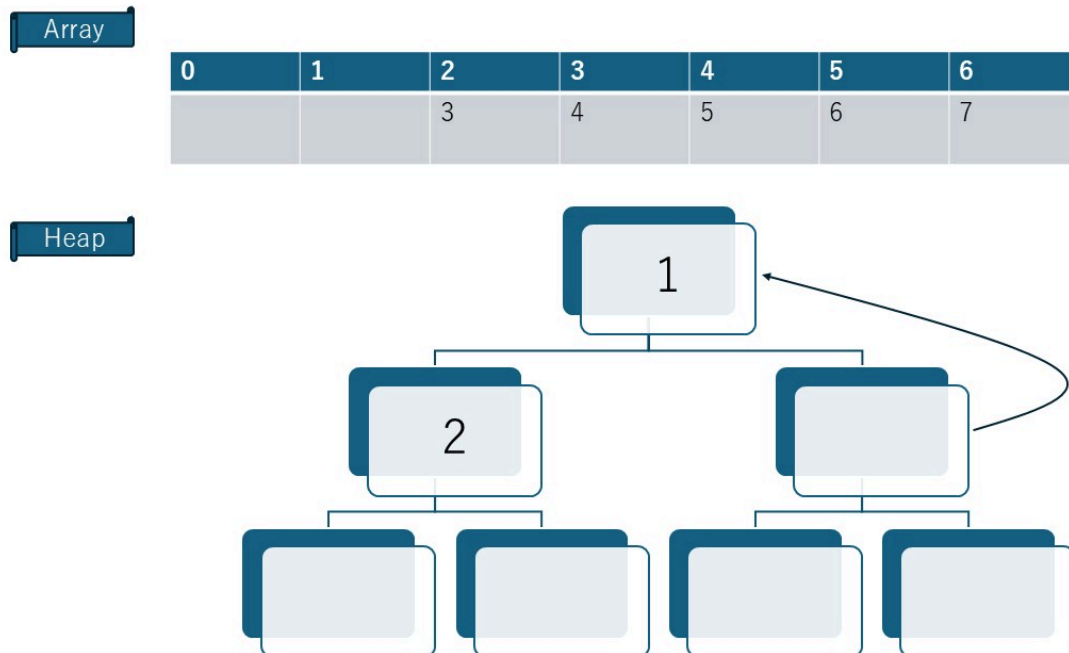
- $1 < 3$ なので、安定



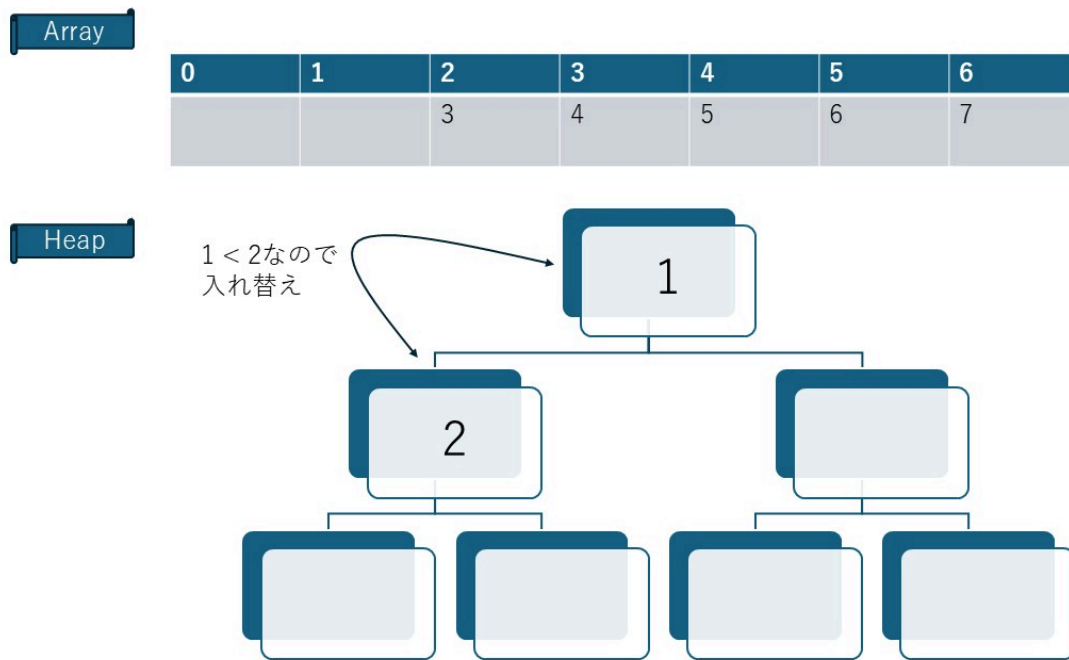
- 頂点の 3 を配列に戻す



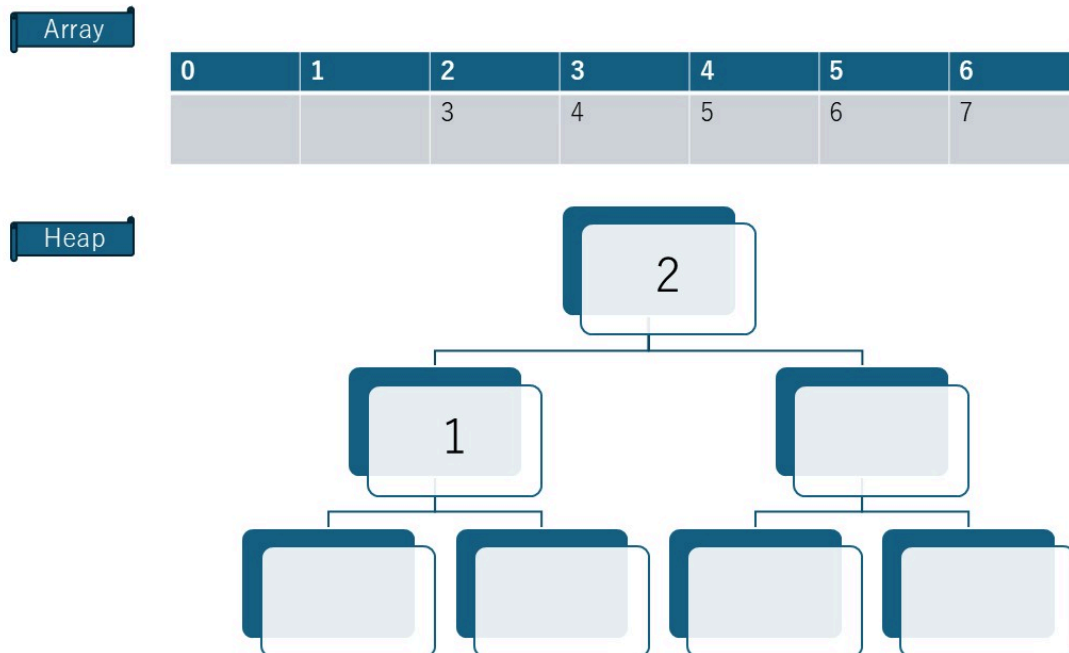
- 右下の 1 を空いた頂点に移動



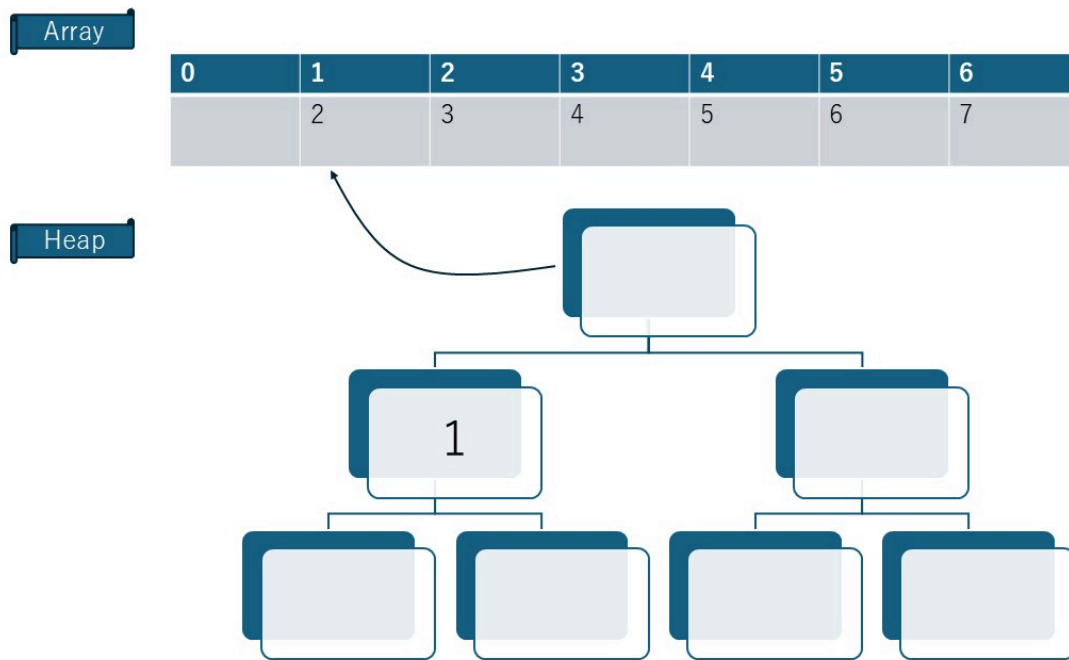
- $1 < 2$ なので、入れ替え



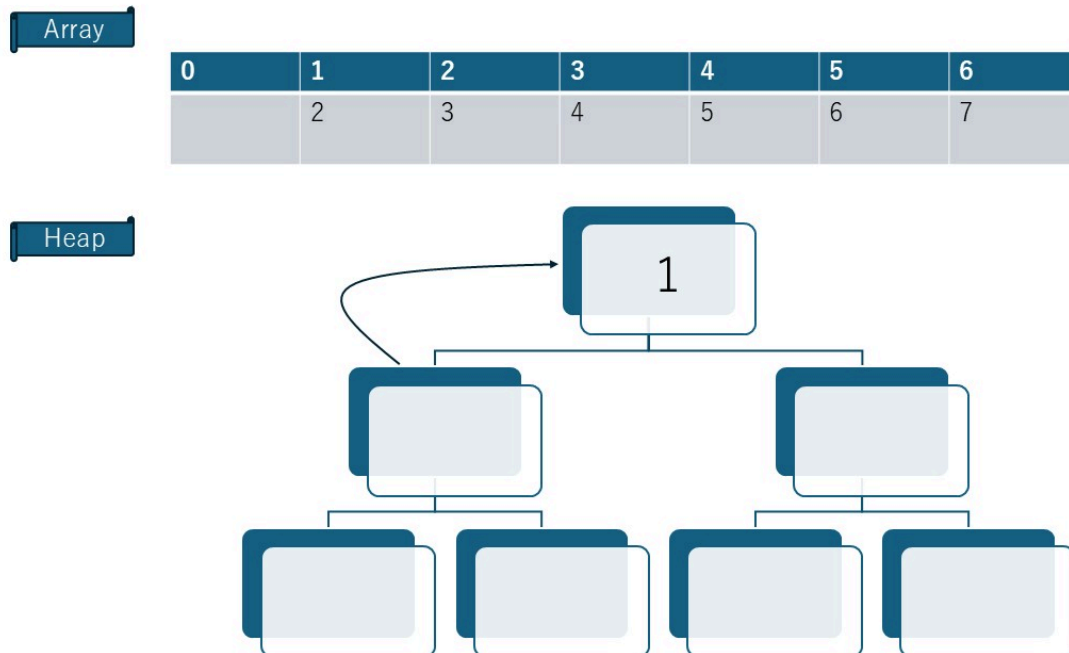
- 安定



- 頂点の 2 を配列にも戻す



- 右下の 1 を空いた頂点に移動



- 頂点の 1 を配列に戻して、完成

